

2026-06-10-SP3-fork-mechanism-plan

SP3 — Fork-ALL CLI agents + auto-merge — Implementation Plan

Field	Value
Revision	1
Created	2026-06-10
Last modified	2026-06-10
Status	active
Maintainer	CLI-Agent Fusion programme
Phase	PLANNING (read-only; no fork created, no submodule swapped, nothing committed or pushed to produce this plan)
Parent roadmap	docs/superpowers/specs/ 2026-06-10-llms-access-master- roadmap.md §SP3 (lines 151-163)
Source analysis	docs/superpowers/specs/ analysis/2026-06-10-E-fork- mechanism.md
Scripts home	scripts/caf/ (LOCAL to HelixCode — operator descoped the constitution submodule 2026-06-10; scripts do NOT go in constitution/scripts/)
Status summary	Decomposition complete; 50 forks (49 top- level + 1 nested); 4 bash files + 3 Challenges; every external mutation OPERATOR-GATED (G-1)
Issues	docs/Issues.md (ATM-NNN assigned at execution kickoff)
Fixed	none yet

Table of contents

- [1. Goal & scope](#)
- [2. Re-verified facts \(live .gitmodules this session\)](#)
- [3. Full fork mapping table \(50 forks\)](#)
- [4. Script architecture \(scripts/caf/\)](#)
- [5. Submodule-swap procedure + buildability validation](#)
- [6. Anti-bluff Challenge per script](#)
- [7. Auto-merge scheduling design \(host-local, NO CI\)](#)
- [8. Config re-export + install + validate tie-in \(→ SP4\)](#)

- [9. Files to create \(all under `scripts/caf/`\)](#)
 - [10. Ordered task list \(RED→impl→GREEN+evidence→rollback\)](#)
 - [11. Operator decisions / gates](#)
 - [12. Risks \(7\)](#)
 - [13. 12-line summary](#)
-

1. Goal & scope

Fork **every** third-party `cli_agents/*` submodule under the `vasic-digital` org with prefix `caf-` (`cli-agent-fork`) → `git@github.com:vasic-digital/caf-<name>.git`, swap each submodule's url in the root `.gitmodules` to our fork, recursively handle nested third-party deps, and keep every fork current by automatically merging upstream `main/master` on a host-local schedule.

Binding constraints (cited, never weakened): - §11.4.113 — absolute no-force-push; merge-onto-latest-main only. Every push to every fork is fast-forward. - §11.4.30 / **CONST-053** — only minimal files (`.gitignore`, governance siblings, nested-`.gitmodules` rewrites) committed to forks; never a blanket `git add -A`; never upstream application-code edits. - §11.4.28(B) (**CONST-051**) — forks stay decoupled / project-not-aware; the ONLY edits we make to a fork are non-source remote-config / `gitignore` / governance-pointer / nested-`.gitmodules`-SSH-rewrite. - **Rule 1 (No CI/CD)** — NO `.github/workflows`, `.gitlab-ci.yml`, etc.; auto-merge is host-local (`launchd/cron`) only. - **Rule 3 (SSH only)** — every fork URL and rewritten nested URL is `git@... SSH`; the one HTTPS upstream (`cline-bench`) becomes SSH in our fork. - §11.4.101 — every irreversible external-mutation step (fork creation, push) is OPERATOR-GATED behind **G-1**; the agent makes only reversible local decisions autonomously. - **CONST-044 (§13.1)** — CONTINUATION updated in the same commit as any state-advancing change.

Out of scope: `cli_agents_resources/*` (resources, not agents); `cli_agents/claude-code-source` (already an own GitLab mirror); de-bluffing the agent bridge providers (that is SP4).

2. Re-verified facts (live `.gitmodules` this session)

Re-parsed the root `/Volumes/T7/Projects/HelixCode/.gitmodules` and the one nested manifest this session (not copied from analysis-E):

- `git config -f .gitmodules --get-regexp 'cli_agents/.*\url' | wc -l` → **50** url entries under `cli_agents/`.
- `submodule.cli_agents/claude-code-source.url` → `git@gitlab.com:milos85vasic/ccode-private.git` → **own/private GitLab mirror** → **SKIP** (default-excluded). ⇒ **49** top-level third-party forks.
- `cli_agents/cline/.gitmodules` exists and contains exactly ONE entry:


```
[submodule "evals/cline-bench"]
  path = evals/cline-bench
  url = https://github.com/cline/cline-bench.git
```

→ **HTTPS** third-party nested dep → fork to `caf-cline-bench`, rewrite to SSH (Rule 3). ⇒ +1 nested fork.

- **Fork count total = 50** (49 top-level + 1 nested).
- Spot-checked verbatim URLs: `cli_agents/codex` → `git@github.com:openai/codex.git`; `cli_agents/warp` → `git@github.com:warpcoddev/Warp.git`; `cli_agents/zeroshot` → `git@github.com:covibes/zeroshot.git`; `cli_agents/x-cmd` → `git@github.com:x-cmd/x-cmd.git`; `cli_agents/vtcode` → `git@github.com:vinhnx/vtcode.git`. All match the mapping table below.
- `cli_agents_resources/OpenAI-Cookbook` uses the per-machine SSH-alias form `org-14957082@github.com:openai/openai-cookbook.git` — **out of scope** (resources dir), but the URL parser **MUST** normalize `org-NNNN@github.com`: → `github.com`: before deriving `owner/repo` so it is never mistaken for a different host.
- **No nested own-org chains** under `cli_agents/*` (only `cline-bench`, which is third-party) → CONST-051 has no `cli_agents` violation to remediate; the resolver script still enforces the rule generically for future nesting.

3. Full fork mapping table (50 forks)

3.1 Top-level `cli_agents/*` → `vasic-digital/caf-<name>` (49)

#	submodule path	upstream url (verbatim)	fork
1	<code>cli_agents/agent-deck</code>	<code>git@github.com:asheshgoplani/agent-deck.git</code>	<code>git@github.com:vasic-digital/caf-agent-deck.git</code>
2	<code>cli_agents/aiagent</code>	<code>git@github.com:Xiaoccer/AIAgent.git</code>	<code>git@github.com:vasic-digital/caf-aiagent.git</code>
3	<code>cli_agents/aichat</code>	<code>git@github.com:sigoden/aichat.git</code>	<code>git@github.com:vasic-digital/caf-aichat.git</code>
4	<code>cli_agents/aichat-llm-functions</code>	<code>git@github.com:sigoden/llm-functions.git</code>	<code>git@github.com:vasic-digital/caf-aichat-llm-functions.git</code>
5	<code>cli_agents/aider</code>	<code>git@github.com:Aider-AI/aider.git</code>	<code>git@github.com:vasic-digital/caf-aider.git</code>
6	<code>cli_agents/amazon-q</code>	<code>git@github.com:aws/amazon-q-developer-cli.git</code>	<code>git@github.com:vasic-digital/caf-amazon-q.git</code>
7	<code>cli_agents/bridle</code>	<code>git@github.com:jeremylongshore/claude-code-plugins-plus-skills.git</code>	<code>git@github.com:vasic-digital/caf-bridle.git</code>
8	<code>cli_agents/claude-code</code>	<code>git@github.com:anthropics/claude-code.git</code>	<code>git@github.com:vasic-digital/caf-claude-code.git</code>
9			

#	submodule path	upstream url (verbatim)	fork
	cli_agents/claude-squad	git@github.com:smtg-ai/claude-squad.git	git@github.com:vasic-digital/caf-claude-squad.git
10	cli_agents/cli-agent	git@github.com:NathanGr33n/CLI_Tool.git	git@github.com:vasic-digital/caf-cli-agent.git
11	cli_agents/cline	git@github.com:cline/cline.git	git@github.com:vasic-digital/caf-cline.git
12	cli_agents/codai	git@github.com:meysamhadeli/codai.git	git@github.com:vasic-digital/caf-codai.git
13	cli_agents/codename-goose	git@github.com:jgenerali/codename-goose.git	git@github.com:vasic-digital/caf-codename-goose.git
14	cli_agents/codex	git@github.com:openai/codex.git	git@github.com:vasic-digital/caf-codex.git
15	cli_agents/codex-skills	git@github.com:openai/skills.git	git@github.com:vasic-digital/caf-codex-skills.git
16	cli_agents/conduit	git@github.com:lostintangent/conduit-release.git	git@github.com:vasic-digital/caf-conduit.git
17	cli_agents/copilot-cli	git@github.com:github/copilot-cli.git	git@github.com:vasic-digital/caf-copilot-cli.git
18	cli_agents/crush	git@github.com:charmbracelet/crush.git	git@github.com:vasic-digital/caf-crush.git
19	cli_agents/deepseek-cli	git@github.com:holasoymalva/deepseek-cli.git	git@github.com:vasic-digital/caf-deepseek-cli.git
20	cli_agents/deepseek-cli-youkpan	git@github.com:youkpan/deepseek-cli.git	git@github.com:vasic-digital/caf-deepseek-cli-youkpan.git
21	cli_agents/fauxpilot	git@github.com:fauxpilot/fauxpilot.git	git@github.com:vasic-digital/caf-fauxpilot.git
22	cli_agents/forge	git@github.com:antinomyhq/forge.git	git@github.com:vasic-digital/caf-forge.git
23	cli_agents/gemini-cli	git@github.com:google-gemini/gemini-cli.git	git@github.com:vasic-digital/caf-gemini-cli.git
24	cli_agents/get-shit-done	git@github.com:glittercowboy/get-shit-done.git	git@github.com:vasic-digital/caf-get-shit-done.git
25	cli_agents/git-mcp	git@github.com:idosal/git-mcp.git	git@github.com:vasic-digital/caf-git-mcp.git

#	submodule path	upstream url (verbatim)	fork
26	cli_agents/gpt-engineer	git@github.com:AntonOsika/gpt-engineer.git	git@github.com:vasic-digital/caf-gpt-engineer.git
27	cli_agents/gptme	git@github.com:ErikBjare/gptme.git	git@github.com:vasic-digital/caf-gptme.git
28	cli_agents/junie	git@github.com:JetBrains/junie.git	git@github.com:vasic-digital/caf-junie.git
29	cli_agents/mistral-code	git@github.com:Wylgrif/Mistral-code.git	git@github.com:vasic-digital/caf-mistral-code.git
30	cli_agents/multiagent-coding	git@github.com:Danau5tin/multi-agent-coding-system.git	git@github.com:vasic-digital/caf-multiagent-coding.git
31	cli_agents/nanocoder	git@github.com:Nano-Collective/nanocoder.git	git@github.com:vasic-digital/caf-nanocoder.git
32	cli_agents/noi	git@github.com:lencx/Noi.git	git@github.com:vasic-digital/caf-noi.git
33	cli_agents/octogen	git@github.com:dbpunk-labs/octogen.git	git@github.com:vasic-digital/caf-octogen.git
34	cli_agents/open-interpreter	git@github.com:KillianLucas/open-interpreter.git	git@github.com:vasic-digital/caf-open-interpreter.git
35	cli_agents/plandex	git@github.com:plandex-ai/plandex.git	git@github.com:vasic-digital/caf-plandex.git
36	cli_agents/postgres-mcp	git@github.com:timescale/pg-aiguide.git	git@github.com:vasic-digital/caf-postgres-mcp.git
37	cli_agents/qwen-code	git@github.com:QwenLM/qwen-code.git	git@github.com:vasic-digital/caf-qwen-code.git
38	cli_agents/shai	git@github.com:ovh/shai.git	git@github.com:vasic-digital/caf-shai.git
39	cli_agents/snow-cli	git@github.com:MayDay-wpf/snow-cli.git	git@github.com:vasic-digital/caf-snow-cli.git
40	cli_agents/swe-agent	git@github.com:princeton-nlp/SWE-agent.git	git@github.com:vasic-digital/caf-swe-agent.git
41	cli_agents/taskweaver	git@github.com:microsoft/TaskWeaver.git	git@github.com:vasic-digital/caf-taskweaver.git
42	cli_agents/ui-ux-pro-max	git@github.com:nextlevelbuilder/ui-ux-pro-max-skill.git	git@github.com:vasic-digital/caf-ui-ux-pro-max.git

#	submodule path	upstream url (verbatim)	fork
43	cli_agents/vtcode	git@github.com:vinhnx/vtcode.git	git@github.com:vasic-digital/caf-vtcode.git
44	cli_agents/warp	git@github.com:warpdotdev/Warp.git	git@github.com:vasic-digital/caf-warp.git
45	cli_agents/x-cmd	git@github.com:x-cmd/x-cmd.git	git@github.com:vasic-digital/caf-x-cmd.git
46	cli_agents/xela-cli	git@github.com:xelauvas/codeclau.git	git@github.com:vasic-digital/caf-xela-cli.git
47	cli_agents/zeroshot	git@github.com:covibes/zeroshot.git	git@github.com:vasic-digital/caf-zeroshot.git
48	cli_agents/spec-kit	git@github.com:github/spec-kit.git	git@github.com:vasic-digital/caf-spec-kit.git
49	cli_agents/superset	git@github.com:superset-sh/superset.git	git@github.com:vasic-digital/caf-superset.git

3.2 Nested third-party → fork (1)

#	nested path	upstream url (verbatim)	fork	note
50	cli_agents/ cline → evals/cline- bench	https://github.com/ cline/cline-bench.git	git@github.com:vasic- digital/caf-cline- bench.git	HTTPS upstream → Rule 3 ⇒ must be SSH in our fork. After <code>caf-cline</code> exists, rewrite <code>caf-cline</code> 's <code>.gitmodules</code> <code>evals/cline-bench</code> → <code>caf-cline-bench</code> (committed to the fork as a <code>.gitmodules-only</code> minimal change).

3.3 Excluded / special-cased (do NOT fork)

path	url	reason	action
cli_agents/claude-code-source	git@gitlab.com:milos85vasic/ccode-private.git	Already an own private GitLab mirror — not a public third-party upstream.	Skip fork (default - - <code>exclude + caf_is_own_org</code> guard). Re-homing under <code>vasic-digital</code> is out of scope for the fork-all pass.
cli_agents_resources/ OpenAI-Cookbook	org-14957082@github.com:openai/openai-cookbook.git	Resources dir (out of - -	Not forked. Listed only so the parser

path	url	reason	action
		src-dir cli_agents scope); SSH- alias form.	normalizes org- NNNN@github.com: → github.com:.

Naming policy (G-1b, default applied): fork names preserve the existing kebab submodule **directory** name (caf-gpt-engineer, caf-x-cmd), NOT the upstream repo name (which is sometimes CamelCase like Warp, AI Agent, TaskWeaver). This keeps fork [?] submodule traceability and is CONST-052-consistent with the kebab paths already in .gitmodules. No two caf-* names collide across the 50-entry set (verified). Operator may override the prefix via --prefix.

4. Script architecture (scripts/caf/)

Four bash files, house style matched to constitution/scripts/codegraph_update.sh (verified this session): #!/usr/bin/env bash, set -uo pipefail, SCRIPT_DIR/REPO_ROOT resolution, documented exit codes, append-only docs/caf/Status.md ledger, and **anti-bluff PASS that observes real post-state** (the remote, the merge SHA, the git trace) — never just a CLI exit code.

Naming reconciliation (roadmap P3.7): analysis-E proposed caf_fork_all.sh / caf_update_merge.sh / caf_resolve_nested_submodules.sh; the roadmap SP0 list and this task use fork_third_party_submodule.sh / update_fork_from_upstream.sh / resolve_recursive_fork_deps.sh. **This plan adopts the latter (task-specified) names** + a shared caf_lib.sh. All four live at scripts/caf/ LOCAL to HelixCode (operator descoped the constitution submodule — scripts do NOT land in constitution/scripts/).

4.0 Shared parameter surface (all flags / env; NOTHING hardcoded)

```
--org <name>                default: vasic-digital                (env CAF_ORG)
--prefix <str>               default: caf-                      (env CAF_PREFIX)
--src-dir <path>             default: cli_agents                (env CAF_SRC_DIR)
--gitmodules <path>         default: <repo-root>/..gitmodules  (env CAF_GITMODU
--providers <csv>            default: github,gitlab            (env CAF_PROVIDE
--gitlab-group <name>        default: vasic-digital            (env CAF_GITLAB_
--branch <name>              default: "" → auto-detect remote HEAD (main|master
--dry-run                    print planned actions, mutate nothing (DEFAULT in
--only <name[,name]>          restrict to a subset (testing / resumable retry)
--exclude <name[,name]>       skip-list (DEFAULT includes claude-code-source)
--recursive                  follow nested third-party submodules (depth-limite
--depth <n>                  default: 3 (recursion bound)
--workdir <path>             scratch clones dir (default: mktemp -d)
--map-file <path>            default: docs/caf/map.tsv (the name|upstream|fork
```

4.1 caf_lib.sh — shared helpers (sourced by all three)

Pure functions, no side effects beyond logging. Helpers: - caf_normalize_url <url> — strip org-NNNN@, host-normalize (gitlab.com/github.com), strip trailing .git, emit

canonical host/owner/repo. - `caf_is_own_org <norm>` — TRUE if the normalized owner $\in$ {vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory} OR host is the operator's own GitLab namespace (milos85vasic). Used to SKIP (top-level) or flag CONST-051 (nested). - `caf_fork_name <submodule-dir-name>` — `<CAF_PREFIX><dir-name>` (kebab dir name, per §3.3 policy). - `caf_detect_default_branch <remote-or-url>` — `git ls-remote --symref <x> HEAD` $\rightarrow$ main|master. - `caf_log <LEVEL> <args...>` — append TS LEVEL args to docs/caf/Status.md + echo to stdout. - `caf_provider_remotes <fork>` — expand --providers csv to the set of fork remote URLs. - `caf_in_csv <needle> <csv>` — membership test for --only / --exclude / --providers.

4.2 `fork_third_party_submodule.sh` — **fork-or-create + remote wiring (G-1)**

Purpose: for every submodule under --src-dir, create `<org>/<prefix><name>` on each provider, seed from upstream, wire remotes (origin=our fork, upstream=original), and record --map-file. Idempotent: re-running detects an existing fork and skips creation.

Drafted body (illustrative — final lands in scripts/caf/):

```
#!/usr/bin/env bash
# fork_third_party_submodule.sh — SP3 fork-all. §11.4.113-safe
#   (empty-target seed only).
# Exit codes: 0 all-handled | 1 partial (some FAIL logged) | 2
#   env problem (gh/gitlab/git missing)
set -uo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
source "${SCRIPT_DIR}/caf_lib.sh"
caf_parse_args "$@" # fills CAF_ORG/PREFIX/SRC_DIR/
# PROVIDERS/DRY_RUN/...

command -v gh >/dev/null || { echo "gh CLI required" >&2; exit
2; }
caf_in_csv gitlab "$CAF_PROVIDERS" && { command -v glab >/dev/
null || { echo "glab required" >&2; exit 2; }; }

rc=0
git config -f "$CAF_GITMODULES" --get-regexp "submodule\.$
{CAF_SRC_DIR}/.*\.url" \
| while read -r key url; do
    name="$(basename "$(dirname "$key")")" # cli_agents/
    <name>.url -> <name>
    norm="$(caf_normalize_url "$url")"
    caf_is_own_org "$norm" && { caf_log SKIP own-org
"$name"; continue; }
    caf_in_csv "$name" "$CAF_EXCLUDE" && { caf_log SKIP excluded
"$name"; continue; }
```



```

[ -n "$CAF_ONLY" ] && ! caf_in_csv "$name" "$CAF_ONLY" &&
    continue
fork="$(caf_fork_name "$name")"

if [ "$CAF_DRY_RUN" = 1 ]; then
    caf_log PLAN "gh repo fork $norm --org $CAF_ORG --fork-
        name $fork --clone=false"
    echo "$name $url      git@github.com:$CAF_ORG/$fork.git
        git@gitlab.com:$CAF_GITLAB_GROUP/$fork.git" >>
        "$CAF_MAP_FILE"
    continue
fi

    # (a) GitHub fork-or-create (gh fork keeps the upstream
    link; create+mirror-push fallback for HTTPS-only /
    archived / non-forkable)
if gh repo view "$CAF_ORG/$fork" >/dev/null 2>&1; then
    caf_log EXISTS "$CAF_ORG/$fork"
elif gh repo fork "$norm" --org "$CAF_ORG" --fork-name
    "$fork" --clone=false; then
    caf_poll_fork_ready "$CAF_ORG/$fork"          # async
    fork_readiness (Risk 1)
    caf_log FORKED "$CAF_ORG/$fork"
else
    gh repo create "$CAF_ORG/$fork" --private --disable-wiki
    || { caf_log FAIL create "$name"; rc=1; continue; }
    tmp="$(mktemp -d)"; git clone --mirror "$url"
    "$tmp/.git" \
        && git -C "$tmp/.git" push --mirror "git@github.com:
        $CAF_ORG/$fork.git"
    # empty-target seed: §11.4.113-safe (no existing refs
    overwritten)
    rm -rf "$tmp"; caf_log SEEDED "$CAF_ORG/$fork"
fi

# (b) GitLab mirror of the fork (dual-remote)
if caf_in_csv gitlab "$CAF_PROVIDERS"; then
    glab repo view "$CAF_GITLAB_GROUP/$fork" >/dev/null 2>&1
    \
        || glab repo create "$CAF_GITLAB_GROUP/$fork" --
        private || caf_log FAIL gitlab "$name"
fi

# (c) record mapping for swap + merge
echo "$name $url      git@github.com:$CAF_ORG/$fork.git
    git@gitlab.com:$CAF_GITLAB_GROUP/$fork.git" >>
    "$CAF_MAP_FILE"

```

```
done
exit $rc
```

Anti-bluff / §11.4.113: the mirror-push fallback targets an **empty** fork (no existing refs) → not a force, not a history overwrite. Every `gh/ghlab` call is wrapped; non-zero → `caf_log FAIL + continue` (one bad repo never aborts the batch). Rate-limit handling: exponential backoff on HTTP 403/429 via `gh api rate_limit` polling; resumable with `--only` from the partial `--map-file`. **OPERATOR-GATED (G-1):** any non-`--dry-run` invocation creates irreversible external state and is run only after explicit operator go.

4.3 `update_fork_from_upstream.sh` — **fetch + merge-onto-latest upstream (G-1 for push)**

Purpose: keep every fork current by merging upstream `main/master` INTO the fork, fast-forward-preferred, **never force** (§11.4.113). Drives the auto-merge schedule (§7).

Drafted per-fork loop (over `--map-file`):

```
git clone "$fork_origin" "$wd/$fork"
cd "$wd/$fork"
git remote add upstream "$upstream_url" 2>/dev/null ||
    git remote set-url upstream "$upstream_url"
git fetch --all --prune --tags
base="$(caf_detect_default_branch upstream)"          # main|
    master
git checkout "$base"
# §11.4.113 merge-onto-latest-main: base = latest fork main;
    merge upstream on top.
if git merge --ff-only "upstream/$base" 2>/dev/null; then
    caf_log FF "$fork" "$base"
else
    git merge --no-edit "upstream/$base" \
        || { caf_log CONFLICT "$fork"; caf_record_conflict
            "$fork"; continue; } # NEVER -s ours / -X theirs
    caf_log MERGE "$fork" "$base"
fi
for remote in $(caf_provider_remotes "$fork"); do
    git push "$remote" "$base" || caf_log PUSHFAIL "$fork"
        "$remote"                # fast-forward only
done
```

Flags add `--strategy ff-only|merge` (default merge, union-preserving) and `--push/--no-push` (default `--push`). Conflicts are **never** auto-resolved — a conflicted fork is logged to `docs/caf/conflicts/<fork>_<ts>.md` with the conflicting paths and parked for operator review; the loop continues (zero-idle §11.4.94). Because the fork's `origin/<base>` is always the merge base and the merge commit descends from it, **every push is a fast-forward** → **no force ever needed**. An upstream history-rewrite is logged `UPSTREAM-REWRITE` and parked — never force-resolved.

4.4 resolve_recursive_fork_deps.sh — recursive nested-dep classifier

Purpose: enforce the recursive rule. Inside each fork / checked-out submodule, scan for nested .gitmodules; **third-party** nested deps → fork (delegate to fork_third_party_submodule.sh with --src-dir = the nested manifest's dir) + rewrite the fork's nested .gitmodules to our SSH fork URL; **own-org** nested deps → flag as a CONST-051 violation and emit a PULL_TO_ROOT remediation row (NEVER silently rewrite an own-org chain).

```
scan_for_nested() {                                # dir depth
    local dir="$1" depth="$2"
    [ "$depth" -le 0 ] && return
    [ -f "$dir/.gitmodules" ] || return
    git config -f "$dir/.gitmodules" --get-regexp '\.url$' |
        while read -r key url; do
            norm="$(caf_normalize_url "$url")"
            sub="$(echo "$key" | sed -E 's/^submodule\.//; s/\.url$//')"
```

```
            if caf_is_own_org "$norm"; then
                caf_log CONST051 "$dir" "$url"
                echo "$dir $url      PULL_TO_ROOT" >>
"$CAF_NESTED_REPORT"                # hard report, never auto-
rewritten
            else
                fork="$(caf_fork_name "$(basename "$sub")")"
                echo "$dir $url      FORK:$CAF_ORG/$fork" >>
"$CAF_NESTED_REPORT"

                # delegate fork (idempotent) + SSH-rewrite the fork's
                nested .gitmodules
            fi
        done
    # recurse one level deeper over each checked-out nested
    submodule, depth-1
}
```

For cline: scan finds evals/cline-bench (HTTPS, third-party) → emits FORK: vasic-digital/caf-cline-bench and, after caf-cline-bench exists, rewrites caf-cline's evals/cline-bench URL to git@github.com:vasic-digital/caf-cline-bench.git (SSH per Rule 3), committed to caf-cline as a .gitmodules-only minimal change (§11.4.30-clean). Depth-bounded (--depth, default 3); cycle guard via a visited-set on normalized URLs.

5. Submodule-swap procedure + buildability validation

After forks exist and are seeded (§4.2) and `--map-file` is populated:

1. **Rewrite** `.gitmodules` URLs (idempotent, per-line; NEVER a blanket `git add -A` per §11.4.30):

```
while IFS=$'\t' read -r name upstream fork_gh fork_gl; do
    git config -f .gitmodules "submodule.cli_agents/
        $name.url" "$fork_gh"
done < docs/caf/map.tsv          # skips claude-code-source
                                (never in the map file)
```

2. `git submodule sync --recursive` — propagates the new URLs into `.git/config` and each submodule's `.git/config`. (`git submodule set-url` is the per-entry alternative; `sync` is the bulk path.)
3. **Nested swap**: for `cline`, the nested `evals/cline-bench` URL is rewritten INSIDE the `caf-cline` fork's `.gitmodules` (not the parent meta-repo), then that fork commits the change.
4. **Per-fork buildability validation** (`caf_validate.sh` step — captured-evidence gate, NOT `grep-only`):
 - assert every `cli_agents/*` URL in `.gitmodules` now resolves to `vasic-digital/caf-*` (except the flagged `claude-code-source` skip);
 - `git ls-remote <fork>` succeeds for each (the remote really exists);
 - cross-provider parity: `git ls-remote` tips on GitHub == GitLab per fork;
 - re-init the swapped submodule and run its native build smoke (language-appropriate: `go build ./.../npm ci && npm run build/cargo build/pip install -e .` per fork) — captured stdout under `docs/qa/<run-id>/.A` fork that fails to build is logged `BUILDFAIL` and parked (does not block the others).
5. **Commit discipline**: the `.gitmodules` rewrite + submodule pointer updates land in ONE meta-repo commit; CONTINUATION updated in the same commit (CONST-044). Forks receive ONLY `.gitignore` / `governance` / `nested-.gitmodules` minimal commits — never upstream code edits.

6. Anti-bluff Challenge per script

Each script ships an anti-bluff Challenge that bootstraps a **throwaway** repo / temp dir and asserts the **real** post-state (§107 / §11.4.5 / §11.4.98 fully automated), with evidence under `docs/qa/<run-id>/.A`.

- `fork_third_party_submodule.challenge.sh` — temp dir with a fixture `.gitmodules` pointing at one tiny public repo (e.g. `octocat/Hello-World`) under `--src-dir=fixtures`; **dry-run path** asserts the exact `gh/glab` command lines emitted; **live path** (disposable test org/group) asserts `gh repo view <org>/caf-hello-world` exits 0 AND `git ls-remote git@github.com:<org>/caf-hello-world.git` returns refs AND the upstream remote resolves to the original. Negative case: a fixture entry whose URL is an own-org / GitLab-own URL MUST be

SKIPPED (proves the `claude-code-source` guard). Teardown deletes the test fork. PASS observes the **real remote**, never just the CLI exit code.

- `update_fork_from_upstream.challenge.sh` — create a throwaway “upstream” repo + a “fork” clone in temp; add a commit to upstream; run `update_fork_from_upstream.sh --only fixture --workdir <tmp>;` assert the fork’s main now contains the upstream commit SHA (`git log grep`), assert fork/main is a fast-forward descendant of the pre-merge tip, and assert **no** `--force / --force-with-lease / +ref` appears in the captured GIT_TRACE log (§11.4.113 paired mutation: inject a `--force` → Challenge MUST FAIL). Then plant a conflicting edit on both sides → assert the script logs CONFLICT + parks (does NOT silently resolve).
- `resolve_recursive_fork_deps.challenge.sh` — temp parent with a nested `.gitmodules` containing one third-party + one fake own-org URL; run `--dry-run`; assert the third-party row is classified FORK: and the own-org row PULL_TO_ROOT/CONST051; flip the fixture (move the own-org dep to root) → assert NO CONST051 emitted. Paired §1.1 mutation: make `caf_is_own_org` always-false → the own-org row mis-classifies as FORK: → Challenge FAILs.

7. Auto-merge scheduling design (host-local, NO CI per Rule 1)

NO `.github/workflows`, `.gitlab-ci.yml`, **Jenkinsfile**, or **any pipeline**. The auto-merge runs host-local only.

- **Mechanism:** `update_fork_from_upstream.sh` is the unit of work; it is scheduled by a host scheduler the operator already uses — **launchd** on macOS (`~/Library/LaunchAgents/digital.vasic.caf-update.plist`, `StartCalendarInterval` weekly) or **cron** on Linux (`@weekly`). The plist/cron line is documented in `docs/caf/README.md`; it is NOT committed as a project pipeline and NEVER touches host power state (CONST-033).
- **Cadence:** weekly floor (§11.4.45 / §11.4.80 pattern — same cadence as the existing `codegraph_update.sh/codegraph_sync.sh` automation). On-demand manual invocation always available.
- **Background execution (§11.4.89):** the scheduled run launches detached (`nohup ... > qa-results/caf/update_<ts>.log 2>&1 & +disown`); per-fork flock serializes same-fork invocations while different forks run in parallel. Push fan-out to all providers is fast-forward-only (§11.4.113).
- **Failure surfacing:** conflicts → `docs/caf/conflicts/<fork>_<ts>.md`; push failures → `qa-results/caf/push_failures/<ts>_<remote>.log`; the next tick checks per the zero-idle “no external dependency in-flight” gate (§11.4.87 / §11.4.94). NO auto-resolution; conflicts park for operator review.
- **Status ledger:** every run appends to `docs/caf/Status.md` + `Status_Summary.md`, exported to `.html/.pdf` per §11.4.65 (Docs Chain §11.4.106 where wired).

8. Config re-export + install + validate tie-in (→ SP4)

After the swap (§5) the existing governance/config pipeline is re-run so the new remotes propagate, and the forward-bridge config install (SP4 P4.4) consumes the swapped tree:

1. `./scripts/init-submodules.sh` — re-init against the swapped fork URLs.

2. `./scripts/propagate-governance.sh` — cascade Constitution/CLAUDE/AGENTS into each new fork (each fork receives the CONST-047/051 governance siblings — a permitted minimal commit, §11.4.30-clean).
3. `./scripts/verify-governance-cascade.sh` — confirm the anchors are present in every fork.
4. `install_upstreams` from each fork root **iff** it has an `upstreams/recipe` dir (CONST-056); the fork's origin push URLs fan out to all configured providers.
5. **SP4 hand-off:** SP4's config re-export (`helixagent --generate-all-agents`) + the new filesystem-install + LIVE post-install validation (each installed config drives a real proxied prompt → real result, captured evidence) run against the swapped fork tree. SP3 produces the fork substrate; SP4 consumes it. The `caf_validate.sh` resolve+build gate (§5.4) is the explicit SP3→SP4 readiness boundary — SP4 does not start the bridge install until every `caf - *` fork resolves and builds.

9. Files to create (all under `scripts/caf/`)

File	Purpose	Mutates external state?
<code>scripts/caf/caf_lib.sh</code>	shared helpers (normalize/own-org/fork-name/log/branch-detect) sourced by all three	no
<code>scripts/caf/fork_third_party_submodule.sh</code>	fork-or-create on GitHub+GitLab, wire origin+upstream, record map-file	YES — G-1 gated
<code>scripts/caf/update_fork_from_upstream.sh</code>	fetch + merge upstream main/master into each fork, ff-only push, no-force	YES (push) — G-1 gated
<code>scripts/caf/resolve_recursive_fork_deps.sh</code>	recursive nested-dep classifier (third-party→fork, own-org→PULL_TO_ROOT report)	delegates fork (G-1); report-only otherwise
<code>scripts/caf/caf_validate.sh</code>	post-swap resolve + cross-provider parity + per-fork buildability gate	no (read-only probes)
<code>scripts/caf/fork_third_party_submodule.challenge.sh</code>	anti-bluff Challenge (throwaway repo → assert remotes)	disposable test org only
<code>scripts/caf/update_fork_from_upstream.challenge.sh</code>	anti-bluff Challenge (assert merge SHA, ff-only, no-force in trace)	temp dirs only

File	Purpose	Mutates external state?
scripts/caf/ resolve_recursive_fork_deps.challenge.sh	anti-bluff Challenge (assert FORK vs PULL_TO_ROOT classification)	temp dirs only
scripts/caf/README.md	usage, launchd/cron schedule snippet, exit codes, evidence layout	no
docs/caf/Status.md + Status_Summary.md	append-only run ledgers (.html/.pdf siblings per §11.4.65)	no

The `.gitmodules` URL rewrite + `git submodule sync` (§5) is performed by `caf_validate.sh`'s sibling swap step OR a thin `caf_swap_submodules.sh`; the rewrite edits the meta-repo's tracked `.gitmodules` (one commit, CONST-044) — flagged here as the only SP3 change to the meta-repo itself.

10. Ordered task list (RED→impl→GREEN+evidence→rollback)

Each task: RED (failing assertion on current state) → implementation → GREEN with captured evidence → rollback note. Tasks T3.0–T3.4 are author-time (local, reversible); T3.5–T3.7 are the OPERATOR-GATED external-mutation tasks.

#	Task	RED	GREEN
T3.0	<code>caf_lib.sh</code> helpers	unit RED: <code>caf_normalize_url 'org-14957082@github.com:openai/openai-cookbook.git' ≠ github.com/openai/openai-cookbook</code>	unit GREEN: <code>caf_normalize_url 'org-14957082@github.com:openai/openai-cookbook.git' = github.com/openai/openai-cookbook</code>
T3.1	<code>fork_third_party_submodule.sh</code> + its Challenge	Challenge RED on empty impl: no fork created / <code>claude-code-source</code> NOT skipped	Challenge GREEN: <code>fork_third_party_submodule.sh</code> must create fork / <code>claude-code-source</code> must be skipped
T3.2	<code>update_fork_from_upstream.sh</code> + its Challenge	Challenge RED: fork main lacks upstream SHA; mutation injecting <code>--force</code> must be caught	Challenge GREEN: <code>update_fork_from_upstream.sh</code> must update fork main with upstream SHA

#	Task	RED	GR
T3.3	<code>resolve_recursive_fork_deps.sh</code> + its Challenge	Challenge RED: cline-bench not classified; own-org mis-classified	GR cl be ow org dep pro
T3.4	<code>caf_validate.sh</code> resolve+build gate	RED: validate against current tree FAILs (URLs still upstream)	GR all cro per cap
T3.5 (G-1)	Execute fork-all (49 + cline-bench)	n/a (external)	gh gi for file
T3.6 (G-1)	Submodule swap + nested cline-bench SSH rewrite	RED: <code>.gitmodules</code> still points at upstreams	GR cl = su -- one + C
T3.7 (G-1)	Wire auto-merge schedule + governance cascade	RED: no schedule; forks lack governance siblings	GR ins go ca for sch

11. Operator decisions / gates

OPERATOR-GATED (G-1) — irreversible external state, agent will NOT proceed without explicit go (§11.4.101): - **G-1 (primary):** execute the fork-all of 49 top-level repos + 1 nested (`caf-cline-bench`) under `vasic-digital/caf-*` (T3.5) — creates 50 (×2 providers ≈ 100) external repos. *Maps to roadmap decision-register G-1.* - **G-1-push:** the first non--dry-run `update_fork_from_upstream.sh` run that pushes to forks (T3.7).

Reversible — agent proceeds on these defaults unless operator objects (§11.4.101): - **D-1 Naming:** fork name = `caf-<kebab-submodule-dir-name>` (not upstream CamelCase repo name) for traceability — §3.3. Override via `--prefix`. - **D-2 Visibility:** forks created --private (matches existing private own-org posture). Operator may choose public. - **D-3 Providers:** dual-remote GitHub + GitLab (`--providers github,gitlab`, both under `vasic-digital`). Operator may drop GitLab via `--providers github`. - **D-4 Cadence:** weekly auto-merge (matches codegraph automation). Operator may set `@daily`. - **D-5 Merge strategy:** merge (union-preserving) default vs `ff-only`. Conflicts always park — never auto-resolve. - **D-6 Buildability scope:** per-fork build smoke runs at swap time (T3.4); operator may defer heavy builds (`warp`, `superset`, `amazon-q`) to background batches.

Flagged anomalies (no decision needed, recorded): `claude-code-source` skipped (own GitLab mirror); OpenAI-Cookbook out of scope (resources dir + org-NNNN@ alias normalized).

12. Risks (7)

1. **API rate limits + async fork readiness.** $\sim 50 \text{ forks} \times 2 \text{ providers} \approx 100$ create/fork calls + per-fork clone/fetch. GitHub's fork endpoint is rate-limited AND async — `gh repo view` may 404 briefly after `gh repo fork`. *Mitigation:* exponential backoff on 403/429, `gh api rate_limit` polling, `caf_poll_fork_ready` before wiring remotes, resumable --only retry from the partial map-file.
 2. **Scale / disk / time.** Mirror-clone fallback for non-forkable upstreams (HTTPS-only `cline-bench`, archived repos) plus large repos (`warp`, `superset`, `amazon-q`) is bandwidth/disk heavy. *Mitigation:* --workdir on a roomy volume, --only batching, background execution (§11.4.89), cleanup trap.
 3. **GitHub↔GitLab dual-remote divergence.** Two mirrors of each fork can drift if a push lands on one and fails the other. *Mitigation:* `update_fork_from_upstream.sh` fans out to ALL provider remotes every run + logs PUSHFAIL per-remote; `caf_validate.sh` parity-checks `git ls-remote` tips across providers; ff-only keeps both linear (§11.4.113).
 4. **Recursive depth / hidden / HTTPS nested deps.** Only `cline` nests today (depth 1, HTTPS), but upstreams can add submodules anytime, some via HTTPS (Rule 3). *Mitigation:* --recursive --depth 3 scan, URL normalization to SSH, cycle guard, CONST-051 hard-report for any nested own-org chain.
 5. **Keeping forks' upstream CODE unchanged.** We do not control upstream source and must not diverge it (or merges conflict forever and we lose "track upstream"). *Mitigation:* forks receive ONLY `.gitignore` + governance siblings + nested-`.gitmodules` SSH rewrites (all non-source, §11.4.30/§11.4.28(B)-clean); merge is always upstream→fork (never reverse); conflicts park; merge-onto-latest-main (§11.4.113) keeps pushes ff-only so no upstream history is ever rewritten.
 6. **claude-code-source mis-fork.** A naive "fork everything" would create a pointless `caf-claude-code-source` over the operator's own GitLab mirror. *Mitigation:* `caf_is_own_org` + default --exclude `claude-code-source`; asserted by the §6 Challenge negative case.
 7. **No-CI constraint vs scheduling reliability (Rule 1).** Auto-merge must run reliably without any pipeline. *Mitigation:* host-local launchd/cron only (documented, not committed as a project pipeline), background + detached (§11.4.88/§11.4.89), append-only Status ledger + failure logs the next tick reconciles; never touches host power (CONST-033).
-

13. 12-line summary

1. Re-verified live root `.gitmodules` this session: **50** `cli_agents/*` URL entries (`git config --get-regexp ... | wc -l = 50`).
2. **Fork count = 50:** 49 top-level third-party `cli_agents/*` → `vasic-digital/caf-<name>`, plus **1 nested** (`cline-bench`).
3. **Skipped:** `cli_agents/claude-code-source` → `git@gitlab.com:milos85vasic/ccode-private.git` (own GitLab mirror, default-excluded).

4. **Nested:** `cli_agents/cline/.gitmodules` → `evals/cline-bench`
<https://github.com/cline/cline-bench.git> (HTTPS → SSH `caf-cline-bench`, Rule 3); max fleet depth 1, no own-org nested chains (CONST-051 clean).
5. OpenAI-Cookbook (`org-14957082@github.com:`) is out of scope (resources dir) — parser normalizes `org-NNNN@github.com:` → `github.com:`.
6. **Scripts live LOCAL at** `scripts/caf/` (operator descope'd the constitution; NOT `constitution/scripts/`), fully parameterized (`org/prefix/src-dir/providers/branch/depth` — nothing hardcoded), sharing `caf_lib.sh`.
7. `fork_third_party_submodule.sh` — `gh+glab fork-or-create` under `--org/--prefix`, wire `origin(fork)+upstream(original)`, record map-file; idempotent, rate-limit backoff, empty-target mirror-push fallback (§11.4.113-safe).
8. `update_fork_from_upstream.sh` — fetch + merge upstream main/master INTO each fork, ff-preferred, **never force** (§11.4.113), conflicts parked (never `-s ours/-X theirs`), fan-out push to all providers.
9. `resolve_recursive_fork_deps.sh` — recursive (`--depth 3`) classifier: third-party→fork+SSH-rewrite, own-org→hard CONST-051 PULL_TO_ROOT report (never auto-rewritten).
10. Each script ships an anti-bluff Challenge (throwaway repo → assert real remotes / merge SHA / ff-only / no--force in git trace; evidence under `docs/qa/<run-id>/`); swap = per-line `.gitmodules` URL rewrite + `git submodule sync --recursive` + per-fork buildability gate (`caf_validate.sh`).
11. **Every external mutation (fork creation, push) is OPERATOR-GATED behind G-1** (§11.4.101); only local/reversible steps run autonomously; SP3 hands the swapped fork tree to SP4's config re-export+install+validate at the `caf_validate.sh` boundary.
12. Auto-merge is **host-local launchd/cron only (NO CI, Rule 1)**, weekly, detached/background (§11.4.89); 7 risks tracked (rate-limit/async, scale/disk, dual-remote drift, recursive/HTTPS nesting, keep-upstream-code-clean, mis-fork guard, no-CI scheduling).

Fork count: 50 (49 top-level + 1 nested `cline-bench`; `claude-code-source` skipped).

The 3 scripts: `fork_third_party_submodule.sh`, `update_fork_from_upstream.sh`, `resolve_recursive_fork_deps.sh` (+ shared `caf_lib.sh`), all at `scripts/caf/`.